

Multi-System Data Sync

 systemdesignschool.io/fundamentals/multi-system-data-sync



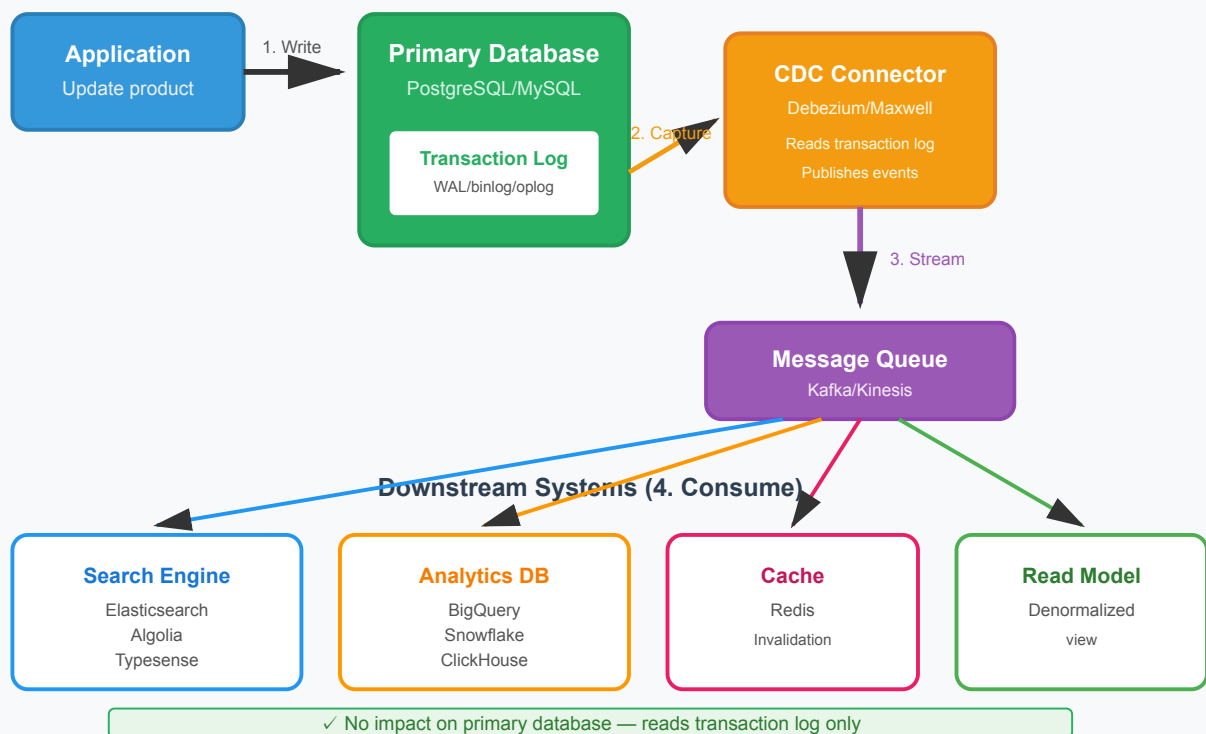
Database-per-service isolates data within service boundaries. But applications often need that data in multiple forms. Users search products while browsing. Analytics teams query purchase patterns. Reports aggregate sales data. Multi-system data sync solves this by streaming database changes to specialized systems without adding load to the primary database.

An e-commerce product database handles transactional writes. Users update inventory, change prices, add new products. These operations require ACID guarantees and fast writes. But product search requires full-text indexing, relevance ranking, and faceted filtering. Running complex search queries directly on the transactional database degrades write performance.

How CDC Works

Change Data Capture monitors the database transaction log. When a row changes, CDC captures the change as an event. This event streams to downstream systems through a message queue or streaming platform. Specialized systems consume these events and update their indexes or tables.

Change Data Capture Flow



The transaction log already records every change for database durability and replication. CDC reads this existing log without adding queries to the database. A product price update writes to the products table and appends to the transaction log. CDC reads the log entry, produces an event with the new price, and publishes to Kafka. A search indexer consumes the event and updates the search index.

Use Cases

Product search benefits from CDC. The primary database stores product data with transactional integrity. The search system uses Elasticsearch, Algolia, or Typesense for full-text search and faceted navigation. CDC keeps them synchronized. When a merchant updates a product description, CDC streams the change. The search index updates within seconds. Users searching for products see current data without querying the primary database.

Analytics pipelines consume CDC events. A data warehouse needs sales transactions for reporting. CDC streams each sale from the transactional database to BigQuery, Snowflake, or ClickHouse. Analysts query the warehouse without impacting the production database. The warehouse stays current within minutes of actual transactions.

Caching layers use CDC for invalidation. When a product price changes, CDC publishes an event. The cache service consumes it and invalidates cached product data. The next request fetches fresh data. This coordinates cache invalidation across distributed cache nodes without polling the database.

Microservice data replication uses CDC. The Order Service needs basic product information for display. CDC streams product changes from Product Service database to Order Service read model. Order Service reads from its local copy instead of making API calls to Product Service. This reduces latency and service coupling.

Implementation Approaches

Log-based CDC reads the database transaction log directly. Debezium reads PostgreSQL's WAL, MySQL's binlog, or MongoDB's oplog. This approach captures every change reliably without modifying application code. The database already maintains the log. CDC just reads it. This minimizes impact on the primary database.

Trigger-based CDC uses database triggers to capture changes. On insert, update, or delete, a trigger writes to a change table. A separate process reads this table and publishes events. This works on databases without accessible transaction logs but adds write overhead. Each change executes the trigger, adding milliseconds to transaction time.

Polling-based CDC queries for changes periodically. A timestamp column tracks last modification. Every minute, query for rows changed since last poll. This approach is simple but misses deletes without a soft-delete column. Polling adds query load to the database. Use it only when log-based CDC is unavailable.

Downstream Systems

Search engines provide full-text search and filtering. Elasticsearch offers distributed search with relevance ranking. Algolia provides hosted search with typo tolerance. Typesense delivers fast open-source search. MeiliSearch focuses on instant search experiences. CDC keeps these indexes synchronized with the source database.

Analytics databases optimize for aggregation queries. ClickHouse handles billions of rows for real-time analytics. BigQuery provides serverless data warehousing. Snowflake separates storage and compute for flexible scaling. CDC streams transactional data to these systems for reporting without impacting production databases.

Materialized views in another database serve read-heavy workloads. The Order Service maintains a read model with denormalized product data from Product Service. CDC updates this read model as products change. Reads query the local read model instead of making API calls.

Caches use CDC events for smart invalidation. Redis caches product details. When CDC publishes a product update event, the cache service deletes that product key. The next read fetches fresh data. This coordinates invalidation across cache nodes without polling.

Trade-offs

CDC provides eventual consistency. The source database commits a change. CDC captures it and publishes an event. The downstream system consumes and processes it. This sequence takes milliseconds to seconds. During this window, the source database shows new data but search results show old data. Most applications tolerate this delay. Users understand that search results might be seconds behind.

CDC adds operational complexity. Deploy and monitor CDC connectors. Handle failures and restarts. Ensure exactly-once or at-least-once semantics. Track lag between source and destination. This infrastructure overhead is justified when it prevents expensive queries against the primary database.

Schema changes require coordination. Adding a column to the source table may require updating CDC configuration, event schema, and downstream consumers. Plan schema migrations across all systems using the data.

Interview Application

When asked to add search to an e-commerce system, explain that running full-text queries against the transactional database degrades performance. Suggest CDC to stream product changes to a search engine like Elasticsearch or Algolia. Mention that CDC reads the transaction log without adding database load.

Discuss eventual consistency explicitly. Search results might lag behind database changes by seconds. For product search, this is acceptable. Users understand search results update periodically. For critical features like inventory counts, consider reading from the source database directly despite the performance cost.

Mention alternative downstream systems. Search engines for full-text search. Analytics databases for reporting. Caches for read performance. Materialized views for denormalized queries. Show awareness that CDC enables multiple specialized systems to consume the same source data.

Explain the operational trade-off. CDC adds complexity but removes expensive queries from the production database. At scale, this trade-off is worthwhile. A system serving 10 million searches per day cannot run those queries against the transactional database.